

# RAPPORT DE STAGE ET DOCUMENTATION TECHNIQUE

*Projet Agent IA RH (RecrutIA)*

---

*Tâche 1 — Module d'Ingestion Intelligente de CV (PDF / DOCX / OCR)*  
*Tâche 2 — Module d'Évaluation IA Agentique & Scoring Sémantique*  
*Tâche 3 — Backend API REST FastAPI, Base de Données & Traçabilité*

|              |                                                                                                                          |
|--------------|--------------------------------------------------------------------------------------------------------------------------|
| Entreprise   | ARTIWEB — Agence Communication & Marketing Digital, Fès, Maroc                                                           |
| Site web     | <a href="https://artiweb.ma">https://artiweb.ma</a>                                                                      |
| Contact      | contact@artiweb.ma   (+212) 664-017447                                                                                   |
| Partenariat  | Google Partner certifié                                                                                                  |
| Projet       | Agent IA RH / RecrutIA — Automatisation intelligente du recrutement                                                      |
| Technologies | Python 3.12   FastAPI   SQLAlchemy   PostgreSQL / SQLite   Pydantic V2  <br>pdfplumber   PyMuPDF   sentence-transformers |

# I. PRÉSENTATION ARTIWEB ET ARCHITECTURE 3 COUCHES

ARTIWEB est une agence de communication et marketing digital basée à Fès (Google Partner). Le projet **RecrutIA** a été conçu avec une architecture modulaire à 3 couches :

| Couche               | Rôle                                                                              | Statut   |
|----------------------|-----------------------------------------------------------------------------------|----------|
| 1 — Ingestion        | Transformer un CV brut (PDF/DOCX/Scan) en dictionnaire JSON structuré sans LLM.   | RÉALISÉE |
| 2 — IA Agentique     | Évaluer correspondance candidat/offre : score 0–100 + justification argumentée.   | RÉALISÉE |
| 3 — Backend API REST | Connecter Tâches 1 & 2 avec base de données (PostgreSQL/SQLite), Swagger & Audit. | RÉALISÉE |

## II. TÂCHE 1 — COUCHE D'INGESTION INTELLIGENTE DE CV

Module Python autonome (ingestion/) :

**Cascade 3 niveaux** : pdfplumber (texte natif) → PyMuPDF (secours) → OCR Tesseract 300 DPI (scans).  
Fichiers Word via python-docx.

**Nettoyage Unicode NFC** : Suppression des ligatures PDF (fi, fl), icônes (•, ★) et fusion des mots coupés en fin de ligne.

**Parsing NLP** : Matching competences\_ref.json, fusion des dates d'expérience et hiérarchie diplômes (PhD > Master > Licence > BTS).

**Validation** : 21/21 tests automatisés PASSED en 0.91s.

## III. TÂCHE 2 — COUCHE IA AGENTIQUE & SCORING SÉMANTIQUE

Module Python autonome (ia\_agentic/) :

**Filtre Critères Durs** : Élimination immédiate (Score 0.0, statut rejete\_auto\_filtre) si expérience < minimum ou compétences obligatoires manquantes.

**Embeddings & Cosine Similarity** : Vectorisation all-MiniLM-L6-v2 + score combiné (40% vectoriel + 40% compétences + 20% expérience).

**Justification LLM anti-hallucination** : Groq API / Ollama / Fallback déterministe basé strictement sur les faits du CV.

**Validation** : 7/7 tests automatisés PASSED sur 3 scénarios (bon match, rejet dur, match modéré).

## IV. TÂCHE 3 — BACKEND FASTAPI, BASE DE DONNÉES & TRAÇABILITÉ

### 4.1 Objectif de la Tâche 3

Développer l'API REST backend (backend/) qui orchestre les Couches 1 et 2 au sein d'un système fluide, persistant et auditable en base de données.

### 4.2 Comment Exécuter et Tester la Tâche 3

#### Commande A — Démarrer le serveur API Backend (FastAPI + Swagger UI) :

```
cd C:\Users\oo\OneDrive\Bureau\agent_ai
python run_backend.py
```

Accès Swagger UI Documentation : <http://127.0.0.1:8000/docs>

#### Commande B — Lancer la suite de 35 tests automatisés (Tâches 1, 2 & 3) :

```
python -m pytest tests/ -v
```

### 4.3 Architecture des Fichiers Backend (backend/)

```
agent_ai/
├── backend/
│   ├── __init__.py           # Export principal
│   ├── database.py          # SQLAlchemy DB Config (PostgreSQL / SQLite fallback)
│   ├── models.py            # ORM Tables: Offre, Candidat, Candidature, AuditLog
│   ├── schemas.py           # Pydantic V2 Schemas pour requêtes/réponses API
│   ├── pipeline.py          # Connecteur end-to-end (Tâche 1 → Tâche 2 → DB)
│   └── main.py              # Application FastAPI + endpoints REST
├── tests/
│   ├── test_ingestion.py    # 21 tests Couche 1
│   ├── test_ia_agentic.py   # 7 tests Couche 2
│   ├── test_backend.py      # 7 tests d'intégration API REST FastAPI (TestClient)
│   └── run_backend.py       # Script d'exécution Uvicorn
```

### 4.4 Endpoints API REST Exposés

| Méthode & Route                       | Description                                                                                     | Traçabilité Audit   |
|---------------------------------------|-------------------------------------------------------------------------------------------------|---------------------|
| POST /api/offres                      | Création d'une offre d'emploi RH avec ses critères (exp min, compétences).                      | OFFRE_CREEE         |
| GET /api/offres                       | Lister toutes les offres d'emploi enregistrées.                                                 | Lecture             |
| POST /api/candidatures                | Upload du CV (.pdf/.docx) + offre_id. Déclenche le pipeline automatique Tâche 1 → Tâche 2 → DB. | CANDIDATURE_SOUMISE |
| GET /api/offres/{id}/candidatures     | Lister les candidatures d'une offre, TRIÉES PAR SCORE DÉCROISSANT.                              | Consultation RH     |
| GET /api/candidatures/{id}            | Consulter la fiche détaillée d'une candidature (score, justification, JSON brut).               | Consultation RH     |
| PATCH /api/candidatures/{id}/decision | Enregistrer la décision RH (VALIDE, CORRIGE, REJETE) avec note et utilisateur.                  | DECISION_RH_..      |
| GET /api/audit                        | Consulter l'historique complet d'audit de toutes les actions système et RH.                     | Audit & Compliance  |

### 4.5 Résultats des Tests Backend & Validation Globale

### ■ RESULTAT TÂCHE 3 — 7 TESTS FASTAPI PASSED (tests/test\_backend.py)

test\_01\_health\_check : GET / (Statut API 200 OK)

test\_02\_creer\_offre : POST /api/offres (Offre créée et ID retourné)

test\_03\_lister\_offres : GET /api/offres (Listing d'offres)

test\_04\_soumettre\_candidature : POST /api/candidatures (Upload PDF + Ingestion → Scoring → DB)

test\_05\_lister\_candidatures\_triees: GET /api/offres/{id}/candidatures (Tri par score décroissant)

test\_06\_enregistrer\_decision\_rh : PATCH /api/candidatures/{id}/decision (Décision VALIDE + Note)

test\_07\_consulter\_audit\_log : GET /api/audit (Logs d'actions d'audit)

RÉSULTAT BACKEND : 7/7 PASSED — 100% de réussite

### ■ RÉSULTAT GLOBAL DU PROJET — 35 TESTS AUTOMATISÉS PASSED

tests/test\_ingestion.py (Tâche 1) : 21 / 21 tests PASSED

tests/test\_ia\_agentic.py (Tâche 2) : 7 / 7 tests PASSED

tests/test\_backend.py (Tâche 3) : 7 / 7 tests PASSED

TOTAL DU SUITE PROJET : 35 / 35 TESTS PASSED — 100% — Exec: 3.69 secondes

## V. CONCLUSION

Les **\*\*Tâches 1, 2 et 3 sont 100% finalisées, intégrées et validées par 35 tests automatisés\*\***. Le serveur API REST FastAPI fonctionne de bout en bout et offre les interfaces documentées Swagger UI pour le futur Dashboard (Tâche 4).